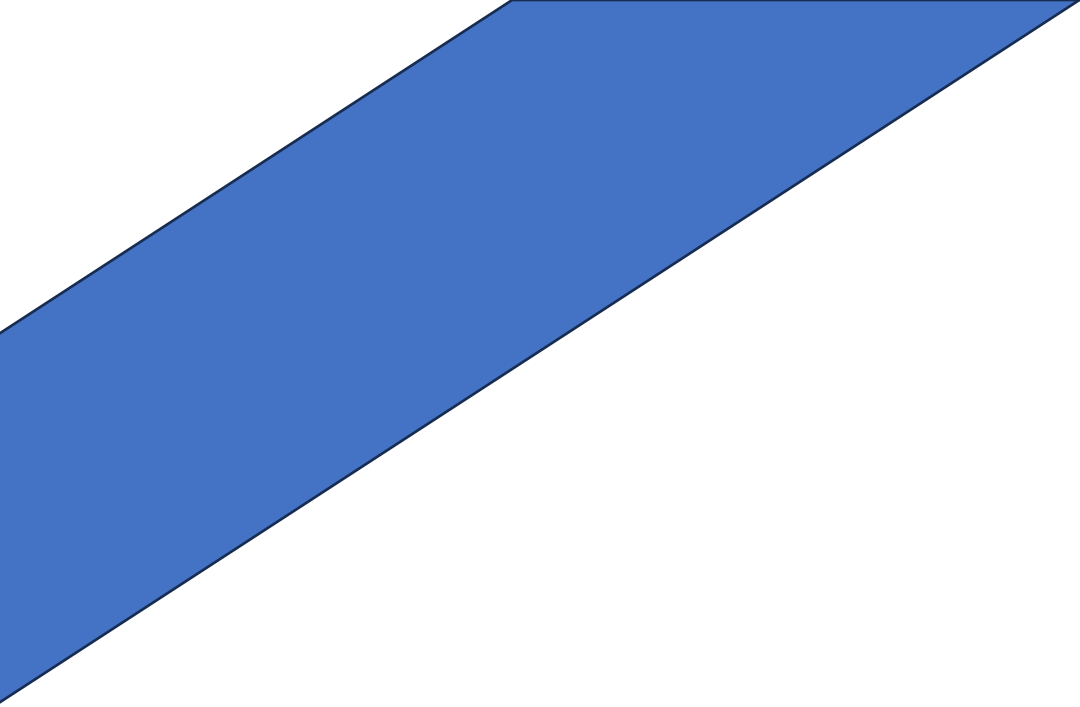




HEPHAESTUS

TravelMind AI – Agent de voyage intelligent

Document de spécifications techniques v1.0

Epitech Lyon – MSc 1 Management des Systèmes d'Information

1. Vision du projet

1.1 Concept

TravelMind AI est un agent de voyage intelligent qui, à partir de quelques informations fournies par l'utilisateur (destination, durée, budget, préférences...etc), génère une expérience de voyage complète avec une planification jour par jour et heure par heure. L'objectif est de dépasser l'expérience d'un simple chatbot : l'agent décide, analyse, interroge ses connaissances, et va chercher des données en temps réel uniquement si nécessaire.

1.2 Ce qui rend ce projet unique

- Planification ultra-détaillée heure par heure, pas seulement des suggestions génériques
- Interaction itérative : l'utilisateur peut valider, rejeter ou demander une alternative pour chaque lieu
- Bouton de réservation simulé (UI complète, fonctionnalité hors scope) pour un rendu réaliste
- Page de découverte de lieux avec cards filtrables (hôtels, restaurants, musées, sites touristiques)
- Architecture agentique réelle : KB + décision + scraping ciblé, pas un wrapper de chatbot

1.3 Pages de l'application

Page	Description
/	Landing page vitrine - présentation de TravelMind AI, CTA vers le chatbot
/explore	Page de découverte - cards de lieux filtrables par catégorie, pays, budget
/chat	Interface de chat - le coeur de l'agent, planification interactive

2. Architecture technique

2.1 Vue d'ensemble

Le backend Python est le cerveau. Le frontend est une interface. Le LLM et les tools sont orchestrés, jamais appelés directement par le front.

L'architecture repose sur 5 couches distinctes avec des responsabilités claires :

Couche	Rôle
Frontend (React)	Interface utilisateur, affichage du chat, cards de lieux, UX
Backend (FastAPI)	Orchestrateur - logique de décision, gestion de la KB, exposition des tools MCP
Knowledge Base	Fichier JSON statique - connaissances métier sur les destinations
MCP / Tools	Fonctions Python exposées - scraping ciblé uniquement si KB insuffisante
LLM local (Ollama)	Raisonnement - Qwen3 ou Mistral via Ollama, aucune API payante

2.2 Flux agentique complet

Voici le flux de décision que l'agent suit pour chaque message utilisateur :

- Réception de la requête utilisateur via /chat (POST)
- Classification de l'intention : sociale (bonjour) → réponse directe | métier → étape 3
- Consultation de la KB (kb.json) — les connaissances statiques sont toujours consultées en premier
- Décision : KB suffisante → réponse directe | données manquantes → appel tool MCP
- Appel tool MCP ciblé (scraping hôtels, vols, météo, attractions) si nécessaire

6. Fusion KB + données scrapées → envoi au LLM pour raisonnement
1. Formulation de la réponse finale - planification structurée renvoyée au frontend

2.3 Structure de la Knowledge Base

La KB est un fichier kb.json chargé au démarrage du backend. Elle contient les connaissances statiques sur les destinations : périodes idéales, climat, attractions principales, conseils pratiques, budget moyen. Elle est consultée avant tout appel externe.

```
Exemple : { "destinations": { "Japon": { "periodes_ideales": ["mars", "avril", "octobre"], "budget_moyen_jour": "100-150€", "incontournables": ["Tokyo", "Kyoto", "Osaka"], "conseils": "Réserver les hébergements 3 mois à l'avance" } } }
```

3. Stack technique

3.1 Technologies imposées et choisies

Composant	Technologie	Statut / Justification
Frontend	React 18 + TypeScript	Recommandé par le sujet - typage fort, composants réutilisables
Styling	Tailwind CSS + shadcn/ui	Développement UI rapide, design system cohérent
Backend	Python 3.11 + FastAPI	Recommandé - async natif, auto-doc Swagger, idéal pour orchestration
LLM Runtime	Ollama (local)	Imposé - modèle open-source local obligatoire
Modèle LLM	Qwen3:4b ou Mistral:7b	Qwen3 testé et validé par le sujet, Mistral en fallback
MCP / Tools	Python pur (FastAPI routes)	Simulation MCP via endpoints structurés - pas de lib externe requise
Scraping	BeautifulSoup4 + httpx	Léger, rapide, suffisant pour scraping ciblé non-JS
Scraping JS	Playwright (si besoin)	Pour sites nécessitant JS (Booking, Airbnb)
KB	JSON statique (kb.json)	Chargé au démarrage du backend, consulté en priorité
Gestion état chat	React Context + useReducer	Gestion de l'historique de conversation côté frontend

3.2 Outils de développement

Outil	Usage	Lien
Git + GitHub	Versioning, branches, PR	github.com
VS Code	IDE principal recommandé	code.visualstudio.com
Postman	Test des endpoints API	postman.com
Swagger UI	Auto-généré par FastAPI doc API interactive	/docs (local)
Ollama	Runner LLM local	ollama.ai
Docker	Containerisation backend + Ollama	docker.com
Notion / Trello	Gestion des tâches et suivi du projet	À définir en groupe

4. Structure du projet

4.1 Organisation des fichiers

Dossier / Fichier	Contenu
hephaestus/	Racine du projet
├── frontend/	Application React + TypeScript
├── src/components/	Composants UI (Chat, Cards, Navbar...)
├── src/pages/	Pages : Home, Explore, Chat
├── src/hooks/	Custom hooks React
└── src/types/	Types TypeScript partagés
├── backend/	Application FastAPI Python
├── main.py	Point d'entrée FastAPI + routes
├── agent/	Logique agentique (décision, orchestration)
├── tools/	Tools MCP (scraping, traitement données)
├── kb/	Knowledge Base (kb.json + loader)
├── llm/	Client Ollama + prompt templates
└── models/	Schémas Pydantic (requêtes/réponses)
├── docs/	Documentation, diagrammes d'architecture
└── README.md	Installation, lancement, présentation du projet

5. Outils MCP — détail des tools

5.1 Liste des tools prévus

Règle d'or : un tool ne doit être appelé que si la KB ne contient pas l'information. Chaque appel externe doit être justifié par un besoin informationnel réel.

Tool	Données récupérées	Source scraping
destination_info_tool	Infos générales sur une destination (si absente de la KB)	Wikipedia / Lonely Planet
hotel_search_tool	Hôtels disponibles, prix, notes	Booking.com (scraping ciblé)
flight_search_tool	Vols disponibles, prix, durée	Google Flights / Skyscanner
weather_tool	Météo actuelle et prévisions 7 jours	wtrr.in (API publique gratuite)
attraction_tool	Sites touristiques, horaires, prix d'entrée	TripAdvisor / Google Maps
restaurant_tool	Restaurants recommandés, cuisine, prix moyen	TheFork / TripAdvisor

Note : chaque tool retourne un objet JSON structuré et filtré – jamais le HTML brut. Le LLM reçoit uniquement les données essentielles pour son raisonnement.

5.2 Format d'un tool (exemple)

Chaque tool suit ce contrat input/output :

Champ	Valeur (exemple hotel_search_tool)
Input	{ "destination": "Tokyo", "checkin": "2026-03-20", "checkout": "2026-03-25", "budget_max": 100 }

Output	<pre>{ "hotels": [{ "name": "Shinjuku Hotel", "price": 85, "rating": 4.2, "location": "Shinjuku" }], "scraped_at": "2026-01-10T..." }</pre>
Erreur	<pre>{ "error": "scraping_failed", "message": "Site inaccessible", "fallback": "KB utilisée" }</pre>

6. Frontend – spécifications UI/UX

6.1 Page Chat – fonctionnalités clés

- Formulaire d'initialisation : destination, dates, budget, type de voyage (solo, couple, famille), préférences (culture, gastronomie, aventure, détente)
- Affichage de la planification en cartes jour par jour, expandables pour voir le détail heure par heure
- Pour chaque lieu proposé : bouton Valider, bouton Rejeter (avec demande d'alternative), bouton Détails
- Bouton "Réserver tout" simulé — déclenche une animation de confirmation mais aucune réservation réelle
- Indicateur de progression visible pendant que l'agent réfléchit (quel tool est appelé, quelle source est consultée)
- Historique de conversation maintenu dans la session

6.2 Page Explore – fonctionnalités clés

- Cards de lieux avec photo, nom, catégorie, note, pays, budget estimé
- Filtres : catégorie (hôtel, restaurant, musée, site touristique, activité), pays/région, budget
- Données provenant de la KB statique – pas de scraping sur cette page
- Responsive design – mobile first

6.3 Landing Page

- Hero section avec proposition de valeur claire et CTA vers /chat
- Section features – ce qui différencie TravelMind AI
- Section démo ou aperçu visuel du chatbot
- Footer avec liens

7. API Backend – endpoints

Endpoint	Méthode	Description
POST /chat	POST	Endpoint principal – reçoit un message, orchestre l'agent, retourne la réponse
GET /destinations	GET	Retourne la liste des destinations disponibles dans la KB
GET /places	GET	Retourne les lieux de la KB avec filtres optionnels (category, country)
POST /tools/{tool_name}	POST	Appel direct d'un tool MCP (usage interne backend)
GET /health	GET	Healthcheck – vérifie que Ollama est accessible
GET /docs	GET	Documentation Swagger auto-générée par FastAPI

8. Modèles de données principaux

8.1 Requête chat

```
ChatRequest : { "message": string, "history": [{ "role": "user"|"assistant", "content": string }], "context": { "destination": string, "budget": number, "dates": { "start": string, "end": string }, "preferences": string[] } }
```

8.2 Réponse chat

```
ChatResponse : { "message": string, "itinerary": DayPlan[] | null, "tools_used": string[], "kb_used": boolean, "places_suggested": Place[] }
```

8.3 Plan journalier

```
DayPlan : { "day": number, "date": string, "theme": string, "slots": [{ "time": string,  
"duration_min": number, "place": Place, "notes": string, "status":  
"pending"|"validated"|"rejected" }] }
```

9. Bonnes pratiques & pièges à éviter

À faire

- Toujours consulter la KB avant d'appeler un tool externe
- Filtrer et structurer les données scrapées avant de les envoyer au LLM
- Gérer les erreurs de scraping avec un fallback propre (message utilisateur + utilisation KB)
- Séparer clairement les responsabilités : frontend = UI, backend = logique, tools = données
- Tester chaque tool indépendamment via Postman avant intégration
- Versionner kb.json et documenter sa structure

À éviter

- Scraper à chaque requête sans vérifier la KB d'abord
- Envoyer du HTML brut au LLM — toujours parser et filtrer
- Mettre de la logique agentique dans le frontend
- Ignorer la gestion des timeouts (Ollama peut être lent sur petits modèles)
- Oublier la gestion de l'historique de conversation dans le contexte envoyé au LLM

10. Installation & démarrage

Prérequis

- Node.js 18+ et npm
- Python 3.11+
- Ollama installé — télécharger sur ollama.ai
- Git

Démarrage rapide

2. Cloner le repo : `git clone https://github.com/[org]/hephaestus`
3. Installer Ollama et le modèle : `ollama pull qwen3:4b`

4. Backend : `cd backend && pip install -r requirements.txt && uvicorn main:app --reload`
 5. Frontend : `cd frontend && npm install && npm run dev`
 6. Accéder à <http://localhost:5173> (frontend) et <http://localhost:8000/docs> (API)
-